|epcc|

Project Preparation Report

Jacob Josiah Webber

6 April 2017

1 Introduction

This report details the preparation for a final project for an MSc in High Performance Computing. This project aims to improve upon current techniques used to run 3 dimensional finite-difference time-domain (FTDT) simulations on the GPU. The problem with existing methods is that a bounding box is used to store all points in the simulation. This is inefficient in its use of memory. This project hopes to devise and implement methods that make better use of limited GPU high bandwidth memory.

This report opens with a critical review of two previous dissertations that cover different topics to this project. There will then be a brief background and literature review, which will describe some of the theory behind room acoustics, some alternatives to the FTDT technique and an overview of the state of the art in this field. The next sections will provide a comprehensive plan of action for how the work will be undertaken, including a finalised project proposal, and a work plan and risk analysis. Finally, this report will then detail any initial findings made during the project preparation stage, before concluding.

2 Previous Dissertation Review

This section contains a review of two previous dissertations submitted by other students. The reports selected for review are not chosen to be particularly relevant in their technical content to the topic of this project, but are useful as examples of high quality work at the level required.

2.1 Dissertation 1 - "Fulfilling Solvency II regulations using GPUs" by Ludovic Capelli, 2016

This first dissertation [1] discusses work done to speed up the computation of financial solvency for insurance companies, in line with new EU wide regulations. These regulations involve Monte-Carlo simulations that require a very large amount of computational resource to comply with. The project improved upon the previously state of the art GPU performance by a factor of two and was awarded a distinction level mark.

The topic seems to be a particularly suitable one for a project in High Performance Computing. Present technology in use by industry would take approximately a year to calculate whether an insurance provider is solvent. By this stage, the data fed in will be out of date and the impact of any insolvency may have already been felt. It is, therefore, necessary and appropriate to use higher performance techniques to do these calculations, even if doing so requires the additional programming and hardware costs associated with such methods.

The dissertation outlines in an appropriate amount of detail the background to the regulations and details of insurance insolvency, striking a balance between not being a dissertation on finance but providing adequate background for the reader to understand the nature of and motivation behind the calculations performed. This content is included briefly in the introduction and in more detail in the Literature review section.

A comprehensive description of GPU programming techniques is provided for the reader. This includes a generous amount of well styled diagrams describing the GPGPU programming paradigm in both CUDA and OpenMP 4.0. Particular attention is given to achieving optimal memory usage. This information will be useful for undertaking a project on 3D room acoustics modelling - where memory accesses are the major bottleneck.

There are few criticisms that can be offered of what is a very well written dissertation covering a project that was measurably successful in its outcomes. However, it could be said that some of the discussion of computing basics is a little unnecessary for a dissertation of this level, in particular, the description of a CPU as the "brain" of the computer.

This dissertation is generally well formatted and easy to read, using the standard EPCC template in LaTeX. However, there is perhaps a little too much included in the preface, with a full table of contents, table of figures and index of tables, along with blank pages in between these. This means that in a document of 69 pages, the actual report does not start until the 14th page.

To conclude the review of this report, it is clearly a very strong piece of work. Good results were achieved with a 100% speed up on previous work. While Mr Capelli's dissertation was not selected for review on the basis of its relevance to the this project, the detailed discussion of GPGPU programming and memory management will be an extremely useful background for undertaking future work, even when applied to a com-

pletely different problem.

2.2 Dissertation 2 - "Assessing the Performance of Optimised Primality Tests" by Cameron Curry, 2016

The second dissertation [2] discussed here also achieved a distinction level mark. It analyses the computational work required to determine if large numbers are prime. The dissertation explains the mathematics behind this process in a way that is comprehensible even to those who have no background in number theory, and a clear motivation is given as to why such a calculation might be performed.

The dissertation compared performance of three already existing softwares written for the purpose of primality testing. The three applications tested were Genefer, LLR and OpenPFGW. These programs were run on ARCHER, which is a large Cray supercomputer. This allowed CrayPat - Cray's proprietary performance profiling tool to be used.

This dissertation performs a very thorough examination of performance for existing codes, but, unlike [1], makes no attempt to improve performance. It does, however, justify well the reasons for how well the codes performed. It also gives suggestions as to how these codes could be improved.

In general, this too is a very well written dissertation. It is easy to understand and shows thorough work has been done in evaluating the performance of the subject codes. It could be offered as a criticism that this project did not implement any improvements to the code, but it seems reasonable to argue that this is made up for by the thoroughness of the experiments that were conducted. The results of this experiment could go on to help the PrimeGrid project, which uses donated CPU cycles to find large primes, select a core technology for primality testing.

3 Background and Literature Review

Acoustics is the study of the physical aspects of sound. Sound is our perception of varying pressure with time in the air around us. The way that this sound, or pressure variation, propagates through the air around us can be modelled using a well know differential equation known as the wave equation. In one dimension this is

$$\frac{\partial^2 \mathbf{p}}{\partial x^2} - \frac{1}{c^2}\frac{\partial^2 \mathbf{p}}{\partial t^2} = 0 \tag{1}$$

where $\mathbf{p}$ is acoustic pressure, and c is the speed of sound. This result is derived by, among others, Richard Feynman in [3]. It is trivial to generalise this for 3 dimensional systems, which yields

$$\nabla^2 \mathbf{p} - \frac{1}{c^2}\frac{\partial^2 \mathbf{p}}{\partial t^2} = 0 \tag{2}$$

It should be noted that in both of these equations **p** is the deviation from the ambient air pressure.

Much of the work in the field of acoustics entails finding solutions to this equation in various mediums and bounding conditions. In the case of room acoustics, the medium of vibration is air, so c corresponds to the speed of sound in air. The nature of air is common to all spaces, with very minor changes depending on altitude or humidity. The majority of room specific acoustic features are therefore determined by the boundary conditions. For large rooms with complex boundaries, it is not possible to solve this equation analytically, so numerical solutions are found using the finite-difference time-domain (FDTD) technique.

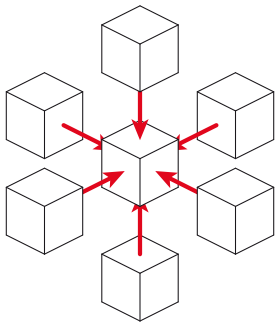

Figure 1: Diagram showing 3 dimensional Von Neumann stencil.
Credit: Wikimedia Commons.

Finite difference methods are a way of approximating and discretising differential equations so that they can be solved numerically. Their use effectively turns the problem of solving a differential equation into a large stencilling operation. A stencilling operation, as shown in figure 1, is the process of replacing each element in a virtual 3D space with the sum of its neighbours.

A simple equivalence between a continuous derivative and a finite difference in one dimensional space is:

$$\frac{df(x)}{dx} \approx \frac{f(x+h) - f(x-h)}{2h} \tag{3}$$

From this, second order finite difference schemes can be derived.

Stencilling operations are in their nature memory bound problems. This is because they involve a lot of data accesses per, relatively simple, calculation. This means that GPUs, with their high bandwidth memory, have been shown to be ideally suited to perform these operations [4]. However, while GPUs have the best performance, they also have

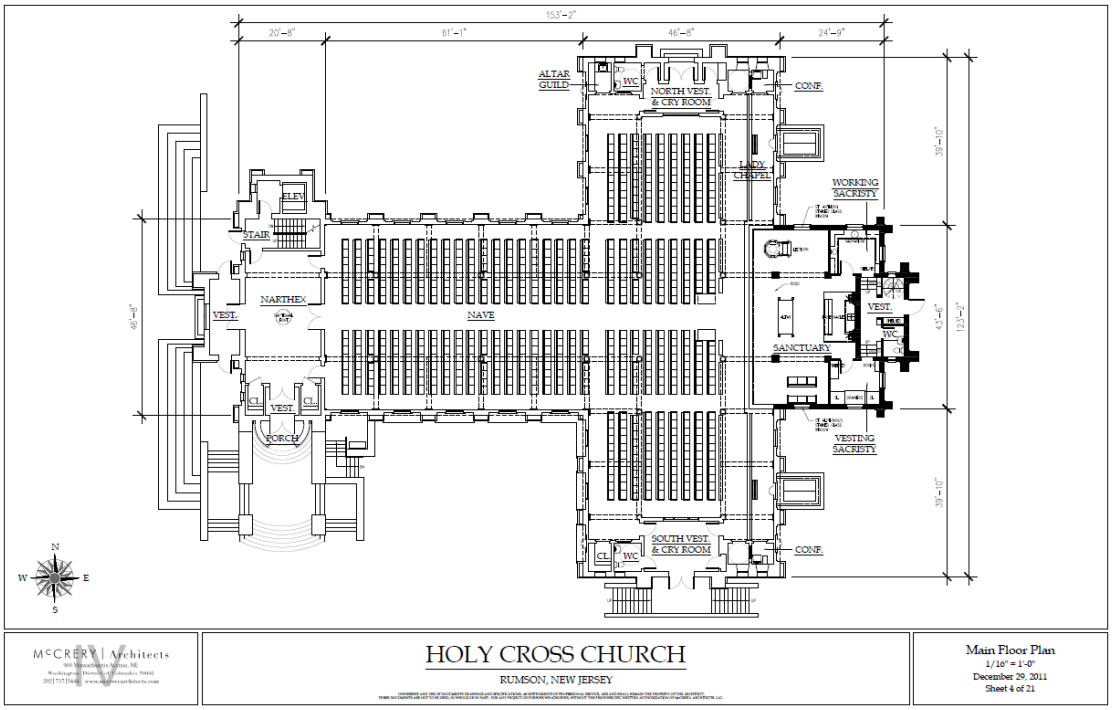

Figure 2: A diagram showing a schematic of a church building.
Credit: McCrery Architects.

limited amounts of memory. This provides problems when trying to do acoustic models as such models require a lot of data points to model even moderately sized rooms. The wavelength of sound at 20kHz, the highest frequency audible to humans, is 17mm. To avoid aliasing or other artefacts, as well as to maintain numerical stability, it is necessary to use multiple points per wavelength.

There is a great deal of literature on the number of points required to be stored for a given simulation, the detail of which goes outside the scope of this report. [5] shows 10s of points per wavelength being used, whereas in previous work by the Acoustics and Audio Group at the University of Edinburgh (AAG) a ratio of$\sqrt{3}$ between time step and sample length has been shown to at least maintain numerical stability. This ratio is known as the Courant limit and a full derivation is shown in [6]. In either case, the memory requirements are large. [7] shows a space of 4775m^2 as requiring 24GB of memory to simulate. In contrast, The Royal Albert hall is popularly claimed to have a volume of approximately 80000m^2.

Another problem with the spacial expense of these computations is caused by the nature of three dimensional arrays. These are cuboid in shape. While this has the benefit of allowing a point's memory address to be easily derived from its spacial location, it does lead to memory being wasted when rooms of less regular shape are being simulated. As an example, figure 2 shows a very common building type: the church. With this shape, a large proportion of the space allocated in memory is assigned to space outside the simulation. Without careful programming, this empty space could also waste compute,

were the stencilling operation to be unnecessarily applied to this region.

Three dimensional arrays are represented in computer memory as long one dimensional arrays. This means that when the stencilling operation is performed many of the neighbouring elements in space are far apart in memory. Non-contiguous device main memory access is much slower than contiguous access. In addition to this, this non-contiguous access increases the number of cache misses as cache lines are loaded in contiguous blocks.

4 Proposal

The purpose of this project is to improve memory usage for irregularly shaped rooms and memory locality in large 3D FDTD acoustics simulations. The proposed method is to replace the large 3D arrays currently used with an array of structs. Each struct will store a smaller sub-array containing points in the simulation, as well as information on the location of neighbouring points to those points of the edge of each sub-array. This will allow blocks where no points are inside the room being simulated to not have to be stored in expensive GPU memory. This technique may also improve data locality. The smaller 3D sub-arrays will result in neighbouring points being scattered less far apart in memory.

Work will be undertaken to compare the performance of this new technique with the current 3D array method, as well as to find optimal values for the shape of the sub-arrays.

5 Work Plan

Task	Start Date	Duration
Implement in 2D and CUDA	25/05/2017	1 week
Extend this to 3D	01/06/2017	1 week
Test performance	08/06/2017	2 weeks
Investigate and carry out extensions	15/07/2017	6 weeks
Write up - including presentation and poster	27/07/2017	3 weeks

Table 1: Table showing formal plan.

As a first step the array of structs will be implemented in two dimensions. This will allow simpler testing of both correctness, as well as providing an initial idea of performance. Performance in three dimensions, however, may be very different. This process will also allow the implementation of an algorithm for converting the bound large array data structure into an array of structs, with only those structs that contain elements

inside the simulation stored. This stage should be done first on the CPU to determine correctness, and then be implemented in CUDA.

The process of implementing in 3D can then be started. This should be a relatively straight forward extension of the first step. Performance data with a range of parameters, including sizes for the sub-array, different shaped room simulations and different GPU models, will then be gathered. If this yields results which are acceptable then work can be done to implement this method in a way that works over multiple GPUs. The AAG has a machine with four NVIDIA k20 devices that could be used for this experiment. Additional performance data should be gathered to see if the proposed methods extend well to multiple GPUs. Performance measurements should include device memory usage, compute time and cache hit/miss ratios.

If time allows after the completion of this work there are many extensions that could be considered. The first of these would be experimenting with different boundary conditions. As discussed in 3, these conditions determined to a very large extent the acoustic characteristics of a room. Work could be done to implement filters of different complexities to model different boundary surfaces. If these filters are computationally expensive work could be done to try to 'hide' this compute by doing it in parallel with memory latencies.

A good amount of time should be allowed for the writing up of a final dissertation. This will contain the following chapters:

- Introduction
- Literature Review
- Methodology
- Implementation Details
- Results
- Review
- Conclusions and future work.

In total there are 12 weeks to complete work on this project. It is proposed that the first week will be spent fully implementing the 2D version in CUDA and the second extending this to 3D dimensions. Weeks 3 and 4 should be spent investigating the performance of this while varying the parameters and hardware. This leaves a number of weeks to investigate extensions to the technique, including some that will be discovered along the way, before the final three weeks are spent writing up. It is hoped that writing will be done along the way to ease the burden towards the end. This writing up period will have to include preparing a presentation as well as designing a poster. Table 1 shows this more formally.

6 Risk Analysis

While there are many risks related to disruptions caused by ill-health and or accident, they are not discussed in detail here as it is assumed these can be dealt with through the relevant university procedures. Instead, risks of a more technical nature are presented.

The most important of these is that it is impossible to achieve good performance using the methods proposed. While it has already been shown that the array of structs reduces memory usage (see section 7), this alone is not enough. GPUs are used because of their relative speed compared to CPUs. If GPU compute time performance using this method is poor, CPUs with their relatively very large amounts of main memory, could be used instead. It must be shown that results from the proposed method are at least comparable to those achieved with current strategies.

Reasons that might cause this lack of performance might be that GPUs are optimised in hardware to operate on standard 3D arrays. CUDA includes the `threadIdx.x`, `threadIx.y` and `threadIdx.y` values to allow optimal striped memory access. It remains to be seen whether breaking with this large 3D array convention will deliver worse, or intolerably bad, performance.

However, as shown in the work plan, there will be plenty of time for fresh investigation after the proposed method has been trialled. In addition, proving this method is unsuccessful is of some merit in itself. It is a subtle philosophical point as to whether a good scientist hopes their theory will be proved correct or not, and one that is not discussed in detail here. While it would be disappointing if this method does not work, it will still be possible to do further work that improves the state of the art in acoustics modelling, and to have disproved what at least initially seems as though it would be a valid theory.

Below follows a more formal risk analysis, where more specific risks are given along with the likelihood, impact, and what will be done to mitigate them.

6.1 Risk 1: Work delayed by bug

Likelihood: Medium

Impact: Medium

It is possible that a bug, even a very simple one, could prevent the code from working and take weeks to fix.

Mitigation: To prevent this happening the code will be thoroughly unit tested from the start. In addition, if it does happen, there will be less time available to investigate and carry out possible extensions. While this is not ideal, it will still be possible to produce a credible dissertation.

6.2 Risk 2: Unable to gain access to AAG machine

Likelihood: Minimal

Impact: Medium

It is assumed there will be access to the AAG's quadruple k20 machine. If this is not possible for whatever reason then it would severely impact the opportunities to do performance experiments

Mitigation: It will still be possible to undertake some experiments on a home desktop equipped with a GTX 970 GPU. In addition to this, it would be possible to use cloud servers equipped with GPUs from a provider such as Amazon Web Services.

6.3 Risk 3: Supervisor unavailable

Likelihood: Minimal

Impact: High

While it would probably be possible to find another, getting them up to speed would cause a delay. There are two supervisors for this project, one from EPCC and the other from AAG. In either case changing supervisor would cause delay.

Mitigation: There is very little that can be done to mitigate for this. However, documenting all work done and keeping an updated wiki can give them the necessary information and help to show that effort has consistently been put in.

7 Initial Findings

As well as investigating and suggesting the new method for 3D acoustics models, some work on these methods' implementations has already been done. This was done in a simpler two dimensional form and only on the CPU.

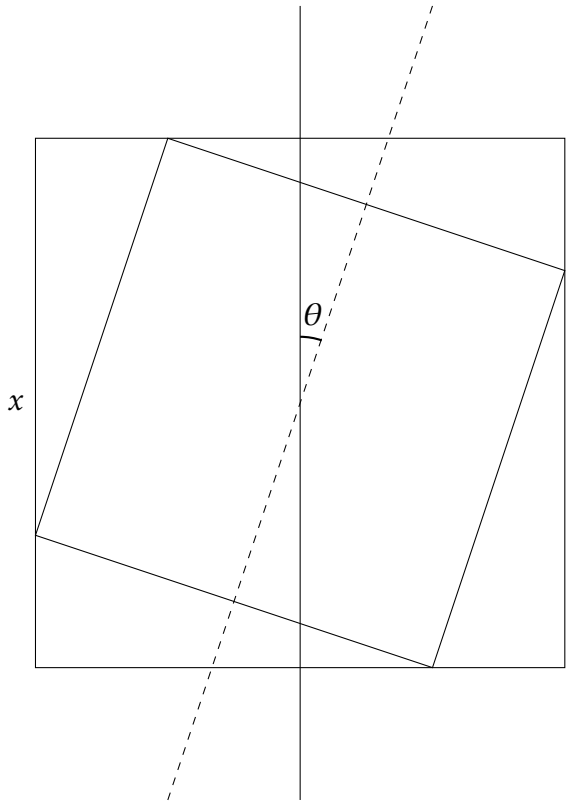

Figure 3: Diagram showing unit square rotated by θ within bounding square, where x is the side of the bounding square. It can be shown that $x = \cos(\theta) + \sin(\theta)$ where $0 < \theta < \frac{\pi}{2}$.

The example chosen to determine the efficiency of memory usage was that of a rotating unit square wholly contained by a bounding square. This arrangement is shown in figure 3. Varying θ also varies the percentage of the space not contained within the unit square. When θ - the angle of rotation of the internal square - is zero, a simple 2D array is perfectly efficient, storing no out of bounds points. This perfect efficiency is not mirrored by the array of structs in this case, as there is additional overhead of storing the location of the neighboring points. When θ is not zero, inefficiencies are introduced by storing values in the bounding square that are not in the internal square. There is useful work to be done in plotting how memory efficiencies change with both θ and the size of the sub-arrays.

The struct shown in listing 1 was implemented in C, along with a function that determines whether a point is within or without the internal unit square. Using the `sizeof()` function it was then shown that such a struct can decrease memory usage when a suitable value of θ is selected.

An algorithm was implemented that converts a series of values in bounded array form into the array of structs form. This was verified using a testing function and proved to be correct.

Initially, instead of integer values storing the indices of neighbouring structs, pointers to

Listing 1: First 2D example

```
struct linked_struct {
    int up;
    int down;
    int left;
    int right;
    float value[N][M];
    float old_value[N][M];
    char inside[N][M];
};
```

neighbouring structs were used. This was determined to be less optimal for two reasons. Firstly, copying these pointers to the GPU will not work as address locations would need to be updated. Secondly, using integers can be more flexible in terms of space usage. For smaller simulations `int32` types can be used, reducing the overhead of storing these within the struct. For larger simulations where the number of structs is too big to be stored within an `int32` variable an `int64` type can be used. This flexibility is not available when pointers are used, as the compiler and hardware determine the size of these variables.

8 Conclusion

To conclude, a new technique has been devised and suggested for the purpose of computing room acoustics simulations. Initial tests have been done that show that it is likely that this method will reduce the amount of memory required for these simulations in some cases. However, work has to undertaken to implement these techniques in three dimensions, as well as to investigate the performance of the stencilling operation in CUDA on this data structure.

While it is a risk that it will not be possible to achieve good performance there is some merit in simply proving this technique is not appropriate. Furthermore, if this is the case there are many other improvements that could be made, and there should still be time to do this.

References

[1] Capeli, L. (2016) *Fulfilling Solvency II regulations using GPUs.* MSc Thesis. University of Edinburgh. Available at: `https://static.ph.ed.ac.uk/dissertations/hpc-msc/2015-2016/Ludovic_Capelli-Dissertation.pdf`

[2] Curry, C. (2016) *Assessing the Performance of Optimised Primality Tests.* MSc Thesis. University of Edinburgh. Available at: https://static.ph.ed.ac.uk/dissertations/hpc-msc/2015-2016/Cameron_Curry-HPC_Dissertation.pdf

[3] Richard Feynman, 1969, *Lectures in Physics, Volume 1,* Addison Publishing Company, Addison

[4] Andreas SchÃd'fer, Dietmar Fey, *High Performance Stencil Code Algorithms for GPGPUs*, Procedia Computer Science, Volume 4, 2011, Pages 2027-2036, ISSN 1877-0509, http://dx.doi.org/10.1016/j.procs.2011.04.221. (Accessed: 5 April 2017)

[5] D. W. Zingy , H. Lomax , and H. Jurgens , *High-accuracy finite-difference schemes for linear wave propagation*, SIAM J. Sci. Comput. 17, 328âĂŞ346, 1996.

[6] Hamilton, B. (2016) *Finite Difference and Finite Volume Methods for Wave-based Modelling of Room Acoustics.* PhD thesis. University of Edinburgh. Available at: http://www2.ph.ed.ac.uk/~s1164563/thesis/hamilton2016phdthesis.pdf (Accessed: 5 April 2017)

[7] Webb, C.J. (2009) *Parallel computation techniques for virtual acoustics and physical modelling synthesis.* PhD thesis. University of Edinburgh. Available at: http://www2.ph.ed.ac.uk/~cwebb2/Site/Docs/CJWebb_thesis_FINAL.pdf (Accessed: 5 April 2017)